

Bài 5: NGOẠI LỆ VÀ XỬ LÝ NGOẠI LỆ

(Số tiết: 5 tiết)

Mục tiêu

- Hiểu được ngoại lệ, xử lý ngoại lệ,
- Vận dụng được ủy nhiệm ngoại lệ với các hàm thành phần của lớp.

Nội dung chính

- Ngoại lệ
- Các lớp ngoại lệ trong Java
- Xử lý ngoại lệ
- Tạo ngoại lệ tùy chỉnh

Nội dung chi tiết

5.1 Ngoại lệ

Ngoại lệ (Exception) trong Java là một sự kiện xảy ra trong khi thực thi chương trình, gây gián đoạn luồng thực hiện bình thường của nó.

Các ngoại lệ xảy ra có thể do lỗi người dùng, lỗi logic, hoặc lỗi hệ thống, ...

Ví dụ 1: chương trình ném ra ngoại lệ do lỗi tính toán logic

```
import java.io.*;
import java.util.*;
public class VD_NgoaiLe1 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        x = inputData.nextInt();
        fx = 10/x;
        System.out.println("f(x) = " + fx);
    }
}
```



Nhập x: 0 ↵
Exception in thread "main" java.lang.ArithmeticException: / by zero
at VD_NgoaiLe1.main(VD_NgoaiLe1.java:10)

Ví dụ 2:

```

import java.io.*;
import java.util.*;
public class VD_NgoaiLe1 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        x = inputData.nextInt();
        fx = 10/x;
        System.out.println ("f(x) = " + fx);
    }
}

```

Chương trình ném
ra ngoại lệ do lỗi
nhập liệu

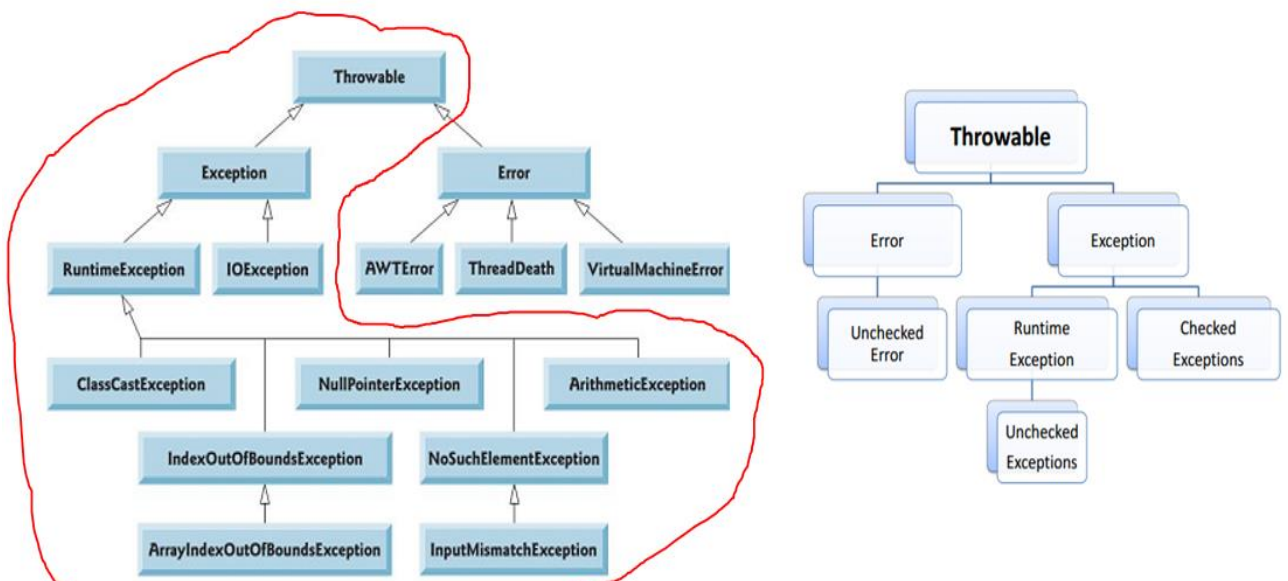
Nhập x: 1K ↵
Exception in thread "main" java.util.InputMismatchException
at java.util.Scanner.throwFor(Unknown Source)
at java.util.Scanner.next(Unknown Source)
at java.util.Scanner.nextInt(Unknown Source)
at java.util.Scanner.nextInt(Unknown Source)
at VD_NgoaiLe1.main(VD_NgoaiLe1.java:9)

5.2. Các lớp ngoại lệ trong Java

Lớp/kiểu ngoại lệ: đóng gói các thông tin & phương thức của ngoại lệ. Tất cả các kiểu ngoại lệ đều dẫn xuất từ lớp *Throwable* hoặc các lớp con của nó. *Throwable* có 2 lớp con trực tiếp:

- Error: Các ngoại lệ nghiêm trọng, chương trình không thể xử lý, ví dụ: *OutOfMemoryError*, *StackOverflowError*, ...
⇒ Error thuộc unchecked exception
- Exception: Các ngoại lệ có thể kiểm soát, xử lý
⇒ CheckedException ?
⇒ UncheckedException?

Tất cả các lớp ngoại lệ trong Java đều dẫn xuất từ lớp ***Throwable***



Hình 5.1 Các lớp ngoại lệ Java

- Ngoại lệ che được, tức bị bỏ qua khi biên dịch, chỉ được check khi thực thi chương trình (**Unchecked exceptions**):
 - ✓ Ngoại lệ này không bắt buộc phải xử lý trong chương trình, nhưng khuyến nghị nên được xử lý
 - ✓ Gồm mọi ngoại lệ thuộc *RuntimeException* và các lớp con
- Ngoại lệ không che được, tức được check ngay khi viết code, biên dịch (**Checked Exceptions**)
 - ✓ Bắt buộc phải xử lý trong chương trình sử dụng *try-catch*, hoặc *throws*, hoặc *throw*



Hình 5.2 Kiểu ngoại lệ Checked và UnChecked.

Bảng 5.1 liệt kê một số kiểu ngoại lệ thường gặp trong chương trình Java:

Lớp Ngoại lệ	Lý do xảy ra ngoại lệ
Exception	Lớp nền của run-time, bắt được các ngoại lệ thuộc các lớp kế thừa
ArithmeticException	Lỗi do tính toán, thường là chia cho 0
ArrayIndexOutOfBoundsException	Lỗi do truy cập đến phần tử có chỉ số vượt quá cỡ mảng

ArrayStoreException	Chương trình lưu trữ dữ liệu không đồng kiểu trên mảng
FileNotFoundException	Truy cập đến một tệp tin không có (theo đường dẫn)
IOException	Lỗi nhập/xuất
NullPointerException	Tham chiếu đến một đối tượng rỗng
NumberFormatException	Lỗi khi chuyển dữ liệu từ xâu sang số
OutOfMemoryException	Không đủ bộ nhớ cấp phát
SecurityException	Exception liên quan đến bảo mật
StackOverfflowException	Tràn stack
StringIndexOutOfBoundsException	Lỗi truy cập đến ô nhớ có chỉ số vượt quá cỡ xâu
ClassNotFoundException	Không tìm thấy lớp
IllegalArgumentException	Đối số truyền vào không hợp lệ
NoSuchMethodException	Sai tên phương thức
EOFException	Lỗi truy cập tệp (đã ở cuối tệp)
AWTException	Ngoại lệ liên quan đến GUI

5.3 Xử lý ngoại lệ

Nếu chương trình KHÔNG xử lý ngoại lệ sẽ gây ra:

- Lỗi lan truyền trong chương trình
- Chương trình kết thúc bất thường
- Tài nguyên không được giải phóng
- Kết quả không mong muốn,

- ...

=> Bắt và xử lý ngoại lệ ntn?

▪ Sử dụng các khối: **try**, **catch**, **finally** và **throw**, **throws**:

- Sử dụng lệnh *try-catch-finally*.
- Ủy nhiệm ngoại lệ với *throws* & *throw*

▪ **Xử lý ngoại lệ - Sử dụng lệnh try-catch-finally**

Sử dụng các khối: **try**, **catch**, **finally** và **throw**, **throws**

~ Sử dụng lệnh *try-catch-finally*.

- **Khối try**: Chứa mã có khả năng gây ra ngoại lệ.
- **Khối catch**:
 - Bắt và xử lý ngoại lệ
 - Có nhiều khối catch cho một khối try
- **Khối finally**
 - Luôn được thực thi bất chấp có xảy ra ngoại lệ hay không
 - Thường dùng để giải phóng/dọn dẹp tài nguyên

Cú pháp câu lệnh try – catch – finally:

```
try {// Khối try  
    <Các câu lệnh có khả năng gây ra ngoại lệ>  
}  
  
catch (<Kiểu ngoại lệ 1> <Tham biến 1>) {// Khối catch 1  
    <Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ 1>  
}  
...  
catch (<Kiểu ngoại lệ n> <Tham biến n>) {// Khối catch n  
    <Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ n>  
}  
  
finally { // Khối finally  
    < Các câu lệnh luôn được thực hiện >  
}
```

Ví dụ 1: chương trình xử lý ngoại lệ dùng lệnh try-catch-finally

```

import java.util.*;
public class VD_NgoaiLe2 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        try {
            x = inputData.nextInt();
            fx = 10/x;
        } catch (ArithmeticException e) {
            System.out.println("Co loi xay ra");
            fx=0;
        }
        finally {
            System.out.println("luôn thực hiện");
        }
        System.out.println("f(x) = " + fx);
    }
}

```

Lỗi này đã được xử lý trong khối catch, chương trình 2 vẫn running

Nhập x: 0 ↵
 Exception in thread "main"
[java.lang.ArithmeticException: / by zero](#)
 at VD_NgoaiLe1.main(VD_NgoaiLe1.java:10)

Ví dụ 2:

```

import java.util.*;
public class VD_NgoaiLe3 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        try {
            x = inputData.nextInt();
            fx = 10/x;
        } catch (InputMismatchException e) {
            System.out.println("Loi nhap sai dinh dang "+e.getMessage());
            fx=0;
        } catch (ArithmeticException e) {
            System.out.println("Loi chia cho 0 "+e.getMessage());
            fx=1;
        }
        finally {
            System.out.println("Luôn thực hiện");
            System.out.println("f(x) = " + fx);
        }
    }
}

```

Lỗi này đã được xử lý trong khối catch, chương trình 3 vẫn running

Nhập x: 1K ↵
 Exception in thread "main" [java.util.InputMismatchException](#)
 at java.util.Scanner.throwFor(Unknown Source)
 at java.util.Scanner.next(Unknown Source)
 at java.util.Scanner.nextInt(Unknown Source)
 at java.util.Scanner.nextInt(Unknown Source)
 at VD_NgoaiLe1.main(VD_NgoaiLe1.java:9)

Lưu ý:

- Các ngoại lệ xảy ra khi gặp lỗi, nếu không bắt và xử lý sẽ làm cho chương trình kết thúc thực thi.
- Khi bắt ngoại lệ, phải biết rõ kiểu ngoại lệ cần bắt. Có thể dùng kiểu cha Exception để thay thế.

▪ Xử lý ngoại lệ - Ủy nhiệm ngoại lệ?

Khi ngoại lệ xảy ra, có 3 cách thức xử lý:

- Bỏ qua (chỉ áp dụng với ngoại lệ che được - unchecked exceptions)
- Xử lý tại nơi xảy ra ngoại lệ: Sử dụng khối lệnh try ... catch ... finally
- Xử lý ở vị trí khác, thời điểm khác trong chương trình => ủy nhiệm ngoại lệ
- Khai báo ngoại lệ với throws:

⇒ Sử dụng khi phương thức có thể ném ra một ngoại lệ, nhưng không xử lý nó

- *Ném ngoại lệ với throw*

⇒ Dùng để ném một đối tượng ngoại lệ trong thân phương thức.

Ủy nhiệm ngoại lệ là ntn?

- Ngoại lệ có thể không cần xử lý tại chỗ mà ủy nhiệm cho mức cao/sau hơn xử lý
- Ở mức nào có phần xử lý ngoại lệ thì ngoại lệ được xử lý tại mức đó
- Mức cao/sau nhất có thể được ủy nhiệm là máy ảo JVM
- Cú pháp:

[<Phạm vi>] [<Kiểm soát truy nhập>] <Tên phương thức> **throws** *ExceptionType* {<Thân phương thức>;};

⇒ Các ngoại lệ **RuntimeException** mặc định được ủy nhiệm cho các mức cao/sau hơn

Ví dụ: Ủy nhiệm ngoại lệ:

```
public class VD_NgoaiLe4 {  
    public static void main(String[] args) {  
        try {  
            divide(10, 0);  
        } catch (ArithmeticException e) {  
            System.out.println("Không thể chia cho số 0!");  
        }  
    }  
    public static void divide(int a, int b) throws ArithmeticException {  
        int result = a / b;  
        (*) System.out.println("Kết quả là: " + result);  
    }  
}
```

Note: Nếu **ExceptionType** thuộc **RuntimeException** có thể không cần sử dụng **throws** để ủy nhiệm. Tuy nhiên, mọi ngoại lệ đều được được nên xử lý ngay = try...catch, hoặc ủy nhiệm.



```
Output - cac_vidu_java_2024 (run) X  
run:  
Không thể chia cho số 0!  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Lệnh (*) bị bỏ qua

Các lợi ích của ủy nhiệm ngoại lệ:

- Dễ sử dụng:
 - o Chương trình dễ đọc và an toàn
 - o Chuyển ngoại lệ đến vị trí có khả năng xử lý
- Tách xử lý ngoại lệ khỏi những đoạn mã thông thường
- Không bỏ sót ngoại lệ
- Gom nhóm và phân loại ngoại lệ

Tung/ném ngoại lệ cưỡng bức

- Phương thức được gọi (called method) có thể tung/ném ngoại lệ cưỡng bức và ủy nhiệm xử lý ngoại lệ đó cho phương thức gọi nó (calling method)
- Thường sử dụng để kiểm soát dữ liệu vào - ra hoặc tại một bước nào đó trong quá trình xử lý dữ liệu
- Cú pháp: Sử dụng từ khóa **throw**:

```
[<Phạm vi>] [<Kiểm soát truy nhập>] <Tên phương thức> throws
ExceptionType {
    <Thân phương thức>;
    throw new exceptionTypeConstruct();
};
```

Ví dụ:

```
public class VD_NgoaiLe5 {
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (IllegalArgumentException e) {
            System.out.println("Ngoại lệ xảy ra: " + e.getMessage());
        }
    }

    public static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Tuổi phải từ 18 trở lên để đăng ký.");
        } else {
            System.out.println("Đăng ký thành công!");
        }
    }
}
```

Lỗi không thuộc kiểu ngoại lệ này, nhưng cưỡng bức ép về loại ngoại lệ này. Ví dụ: Thử cưỡng bức ép về kiểu ArithmeticException, hoặc ArrayStoreException

5.4 Tạo ngoại lệ tùy chỉnh

Chúng ta có thể tự tạo lớp ngoại lệ để phục vụ các mục đích riêng.

Lớp ngoại lệ mới phải kế thừa từ lớp Exception hoặc từ lớp dẫn xuất của lớp này.

Có thể cung cấp hai constructor (toán tử tạo lập)

- Constructor mặc định (không tham số)
- Constructor với tham số String và truyền tham số này cho phương thức khởi tạo của lớp cha

Định nghĩa lớp ngoại lệ

```
//file TuoiKhongHopLeException.java
public class TuoiKhongHopLeException extends Exception {
    // Constructor 1
```



```

public TuoiKhongHopLeException(){
// Construtor 2
public TuoiKhongHopLeException(String message) {
    super(message);
    System.out.println("Ngoại lệ tự tạo!");
}
}

```

Sử dụng ngoại lệ

//file VD_NgoaiLe2.java

```

public class VD_NgoaiLe2 {
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (TuoiKhongHopLeException e) {
            System.out.println("Ngoại lệ xảy ra: " + e.getMessage());
        }
    }

    public static void checkAge(int age) throws TuoiKhongHopLeException{
        if (age < 18) {
            throw new TuoiKhongHopLeException("Tuổi phải từ 18 trở lên
để đăng ký.");
        } else {
            System.out.println("Đăng ký thành công!");
        }
    }
}

```

Lưu ý:

Xử lý ngoại lệ là một phần quan trọng trong lập trình Java.

- Giúp chương trình hoạt động ổn định hơn, duy trì sự thực thi của chương trình ngay cả khi gặp lỗi.
- Giúp lập trình viên phát hiện và khắc phục lỗi dễ dàng hơn.

=> Do đó, sử dụng ngoại lệ đúng cách sẽ nâng cao chất lượng mã nguồn và trải nghiệm tích cực của người dùng.

Tổng kết bài học

Bài học này cung cấp kiến thức cho sinh viên các kiến thức cơ bản về ngoại lệ, nhằm đảm bảo các lỗi xảy ra trong chương trình cần phải được dự đoán và xử lý. Điều này góp phần nâng cao chất lượng chương trình.

Cụ thể, các kiến thức về ngoại lệ được giới thiệu trong bài học này bao gồm:

- Ngoại lệ là gì? Khi nào chương trình Java ném ra ngoại lệ, các kiểu ngoại lệ có thể xảy ra trong chương trình java
- Các lớp ngoại lệ trong Java: Java cung cấp cây phân cấp các kiểu ngoại lệ. Sinh viên cần nắm vững các kiểu ngoại lệ này để dự đoán đúng đối tượng ngoại lệ được chương trình ném ra và xử lý đúng lỗi
- Xử lý ngoại lệ: Các cách xử lý ngoại lệ trong Java, ví dụ: xử lý tại chỗ, hoặc ủy thác việc xử lý ngoại lệ cho các thành phần chuyên trách của chương trình
- Tạo ngoại lệ tùy chỉnh: lập trình viên hoàn toàn có thể tự tạo ra kiểu ngoại lệ tùy chỉnh cho riêng mình để bắt các lỗi mà Java chưa hỗ trợ. Khi tạo kiểu ngoại lệ mới, chúng ta cần lưu ý nó phải kế thừa trực tiếp hoặc gián tiếp từ một lớp ngoại lệ mà Java đã cung cấp sẵn.

Bài tập thực hành

Bài tập 1:

Sử dụng cấu trúc try, và 2 khối catch xây dựng chương trình nhập vào 2 số nguyên a, b và in ra kết quả a/b. Yêu cầu: bắt các ngoại lệ khi nhập a, b và ngoại lệ khi thực hiện phép a/b.

Bài tập 2:

Dùng *throw* để tạo ra ngoại lệ khi nhập tuổi cho nhân viên (không âm và không lớn hơn 62 tuổi)

Bài tập 3:

Xây dựng phương thức *nhapTuoi()* và dùng mệnh đề *throws* để chuyển ngoại lệ ra ngoài hàm và thực hiện bắt và xử lý các ngoại lệ khi gọi hàm *nhapTuoi()*.

Bài tập 4:

Viết chương trình

1. Xây dựng lớp *KhachHang* gồm:

- Các thuộc tính: *Họ tên khách hàng*; *Số điện thoại*; *Địa chỉ*
- Các phương thức:
 - o *Nhập thông tin khách hàng (họ tên, số điện thoại, địa chỉ)*
 - o *Hiển thị thông tin khách hàng*

- Kiểm tra số điện thoại có hợp lệ không (phải đủ 10 số).
- 2. Xây dựng lớp KháchHangVIP kế thừa từ lớp KháchHang, thực hiện ghi đè phương thức kiểm tra số điện thoại để chấp nhận thêm đầu số quốc tế (+84).
- 3. Xây dựng chương trình nhập thông tin khách hàng VIP và hiển thị thông tin của họ. Yêu cầu bắt các ngoại lệ khi nhập số điện thoại không hợp lệ.

Bài tập 5: Viết chương trình máy tính đơn giản, có xử lý ngoại lệ

Yêu cầu chi tiết:

- Nhập hai số nguyên từ bàn phím:
 - + Sử dụng Scanner.
- Hiển thị menu với các lựa chọn:
 - + Phép cộng (+)
 - + Phép trừ (-)
 - + Phép nhân (*)
 - + Phép chia (/)
- Xử lý các lựa chọn của người dùng:
 - + Nếu chia, kiểm tra số chia có bằng 0 hay không. Nếu có, ném ra ngoại lệ và xử lý bằng try-catch.
 - + Nếu không, xuất ra ngoại lệ cho mỗi phép tính.
- Tiếp tục hay kết thúc:
 - + Sau mỗi phép tính, hỏi người dùng có muốn tiếp tục không (Yes/No).
 - + Nếu chọn Yes, quay lại menu. Nếu chọn No, thoát chương trình.

Gợi ý triển khai:

Triển khai case study 1 với các thành phần theo yêu cầu của bài toán: